

Isolating Terminal-Cost Misranking in Latent-Space Planning*

Anonymous authors

Paper under double-blind review

Abstract

Latent world-model planners often select actions by minimizing the distance between a predicted outcome and a goal in representation space. We ask whether this terminal cost can fail even when every candidate outcome is exact. On MetaWorld contact tasks, we replace learned rollouts with simulator execution while holding the encoder and CEM budget fixed. Latent L_2 solves 0/64 push and pick-place episodes using the released DINOv2-based representation, while a physical reference cost solves 64/64 push and 49/64 pick-place episodes. The failure is task-dependent: latent L_2 also fails button-press and window-close but solves drawer-close. On the candidate populations CEM creates, latent- L_2 /physical-cost agreement is weak or absent initially and becomes negative after refitting. In matched CEM branches that differ only in refitting cost, latent- L_2 refitting produces candidates 1.13–2.18 cm worse in task distance. A supervised state readout provides a complementary diagnostic: it has good broad held-out rank agreement but modest elite recovery, both of which degrade further on CEM-generated candidates. A second encoder on OGBench-Cube also exhibits residual misranking after predicted endpoints are replaced by rendered outcomes. These results argue for evaluating planning costs on the candidate distributions created by search, not only on held-out trajectory metrics.

1 Introduction

Visual world models increasingly plan by predicting future representations and minimizing their distance to a goal (Zhou et al., 2025; Terver et al., 2026; Maes et al., 2026b). This approach can fail because the prediction is wrong, because search never proposes a useful action, or because latent distance prefers the wrong outcome. These explanations are easy to conflate: a failed episode alone does not reveal which component failed.

This paper isolates the third possibility. We ask a simple question: *if every candidate outcome is exact, does the latent cost choose the physically better one?* In MetaWorld (Yu et al., 2020), we save the simulator state, execute every candidate action sequence, render its endpoint, and encode that image. The learned predictor is removed, while the representation, goal image, terminal cost, and search budget stay unchanged. Simulator state supplies a physical reference cost and verifies that the same search can solve the task. We evaluate two released MetaWorld planning stacks, DINO-WM and JEPA-WM; both route through the same frozen `dinov2_vits14` encoder once their predictors are bypassed, so the MetaWorld study probes one representation through two released implementations, and a second, independently trained encoder on OGBench-Cube tests whether the pattern is specific to that representation.

The answer is often no. With exact candidate outcomes, latent L_2 fails all 64 push and pick-place episodes for both released planning stacks, whereas the physical cost solves all push episodes and 49 pick-place episodes. Candidate-level measurements explain the gap: the costs rank the same outcomes differently, and repeated CEM updates amplify that disagreement. A second cost based on decoded object and hand positions looks accurate on held-out expert frames but also loses rank agreement on CEM’s candidates. Directly changing

*An LLM-based assistant was used for manuscript restructuring and language editing. The authors verified all scientific claims, analyses, and conclusions.

only the refitting cost produces worse later candidates. Residual true-endpoint misranking also appears with a distinct encoder on OGBench-Cube, although learned prediction error explains part of the total regret there.

Our audit has three steps. First, exact simulator execution asks whether a terminal cost fails without prediction error. Second, it evaluates the ordering that cost induces on the candidates CEM actually creates. Third, matched CEM branches change the refitting cost while holding the initial population, snapshots, and base noise fixed. This identifies whether cost ordering changes later proposals. Across MetaWorld and OGBench-Cube, the audit shows that terminal-cost misranking can survive exact outcome prediction. Stateprobe further shows why average held-out accuracy is not enough: broad rank agreement need not imply reliable elite selection, especially after search shifts the candidate distribution. The drawer-close boundary limits the conclusion to tasks where latent geometry fails to preserve task-relevant ordering.

2 Related Work

Latent planning and goal geometry. DINO-WM plans in the frozen feature space of a self-supervised visual encoder (Zhou et al., 2025); the released MetaWorld stacks evaluated here come from a later study of joint-embedding world models (Terver et al., 2026). LeWorldModel trains a compact encoder and world model end to end and provides the OGBench-Cube checkpoint used in our second study (Maes et al., 2026b;a; Park et al., 2025). Other approaches give latent geometry a behavioral target: contrastive goal-conditioned RL learns similarities for goal reaching (Eysenbach et al., 2022), and temporal- distance learning predicts how many steps separate two states (Myers et al., 2024). Our question is diagnostic rather than constructive: does an already trained geometry rank the candidates encountered during planning in the right order?

Decision-aware models and objectives. Predictive accuracy need not measure usefulness inside a planner. Value-aware model learning weights errors by their effect on value (Farahmand et al., 2017; Farahmand, 2018); value equivalence formalizes when different models support the same planning computations (Grimm et al., 2020); and MuZero and control-oriented model learning train models through decision-relevant targets (Schrittwieser et al., 2020; Nikishin et al., 2022). Recent latent-planning methods similarly learn reachability, trajectory-ordering, or action-consistency objectives (Li et al., 2026b;a; Seo et al., 2026), while latent-geometry methods can amortize planning itself (Nguyen et al., 2026). We complement this work by holding the learned representation fixed and measuring its terminal cost against physical outcomes.

Optimization can expose model and score errors. Policies can exploit learned-dynamics errors (Janer et al., 2019), and model objectives may disagree with control performance (Lambert et al., 2020). Planning can also move action sequences away from the training distribution (Parthasarathy et al., 2025). Even without such structure, selecting the minimum of noisy estimates creates an optimizer’s curse (Smith & Winkler, 2006); under repeated optimization, a proxy can diverge further from its target (Gao et al., 2023; Skalse et al., 2022; Karwowski et al., 2024). IMWM separately highlights limited proposal coverage when environment rollout replaces learned dynamics (Gao et al., 2026). Our experiments score the same candidate outcomes with latent and physical costs, then change only the score used to update CEM. Scoring identical candidate outcomes separates ranking error from prediction error and candidate-set contents; the matched-refit intervention then tests how that ranking affects subsequent proposals, following the broader call to evaluate world models by downstream decisions (Yu et al., 2026).

3 Experimental Design

3.1 Models, tasks, and planning protocol

We evaluate the released `dino_wm_metaworld` and `jepa_wm_metaworld` checkpoints through their public EncPredWM interface (Terver et al., 2026). Both use their released preprocessing and action normalization. Both stacks use the same frozen `dinov2_vits14` encoder; once their predictors are bypassed, they are two released-stack checks of the same representation. The contact tasks are `mw-push` and `mw-pick-place`; `mw-reach` is used only as a free-space control. The goal image is the scripted expert trajectory’s final

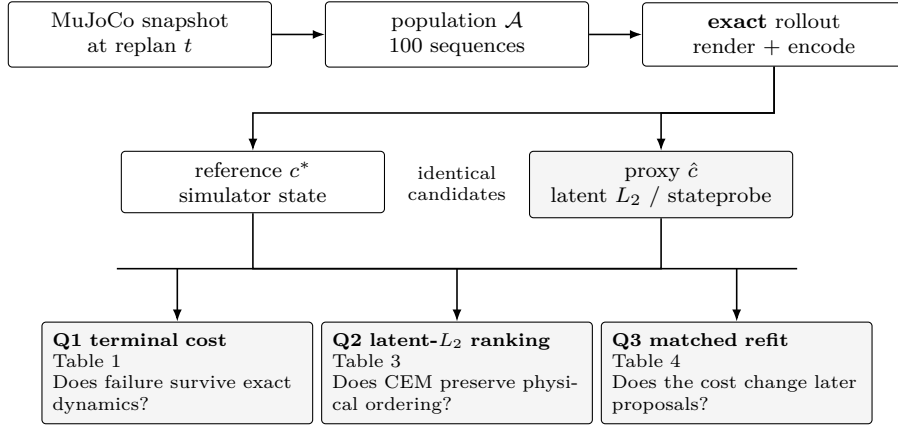


Figure 1: Overview of the controlled audit. Each action sequence is executed from the same simulator snapshot and scored by both the tested cost and a physical reference. We first verify that the search budget can solve the task, then measure latent-cost ranking on its own candidates, and finally intervene on the cost used to update CEM.

frame. Episode success is the simulator’s task flag at the final environment step, not an any-step latch. Its object-to-goal distance radii are 5 cm for push and 7 cm for pick-place.

Unless otherwise stated, the candidate-level experiments use receding-horizon CEM (Rubinstein, 1997) with 100 candidates, a top-10% elite set, six refitting iterations, and action horizon $H = 6$. We denote this budget by 100×6 . The simulator-state cost is a positive control for the planner and budget.

The OGBench study uses the independently trained end-to-end `quentin11/lewm-cube` checkpoint and the official OGBench `cube_single_expert` dataset (Maes et al., 2026b; Park et al., 2025). Its ViT-tiny encoder is distinct from the MetaWorld DINOv2 encoder. We evaluate one checkpoint on one Cube task.

For task breadth, a pre-specified scripted-expert gate retained five additional MetaWorld tasks; peg-insert-side failed this gate. We then ran paired exact-dynamics reference and latent- L_2 arms. Because the released stacks share an encoder, this breadth check uses DINO-WM only. A task is interpreted only when the physical reference establishes finite-budget solvability; all cells, including failed reference controls, are reported. The full eligibility rule and seed ranges are in the supplement.

3.2 Exact candidate execution and three terminal costs

At each replan, the evaluation harness saves the full MuJoCo state. For every action sequence a , it restores the same snapshot, executes a in the simulator, renders the terminal observation, and encodes that observation with the frozen visual encoder. Thus candidate transitions are exact, while the representation and terminal scorer remain unchanged.

We compare three costs:

1. **Latent L_2** , the released Euclidean distance between the encoded endpoint and goal.
2. **Stateprobe**, a supervised diagnostic that decodes object position $\hat{o}$ and end-effector position $\hat{h}$ with supervised probes on frozen endpoint tokens using simulator labels, then applies the planning cost

$$\hat{c}(a) = \|\hat{o}_T(a) - \hat{o}_g\|_2 + 0.5\|\hat{h}_T(a) - \hat{o}_T(a)\|_2. \quad (1)$$

3. **Simulator-state reference**, which applies the same form to simulator object and hand state:

$$c^*(a) = \|o_T(a) - o_g\|_2 + 0.5\|h_T(a) - o_T(a)\|_2. \quad (2)$$

The reference is a shaped physical cost; we also report task success and direct object-to-goal distance. Stateprobe combines the frozen representation with a supervised readout and a fixed functional form, so results below diagnose whether task-relevant state is *accessible through this readout and cost*, not whether the encoder discards it; Appendix D discusses this distinction further.

3.3 Measuring cost quality during search

For each CEM population, we compare the learned and physical costs by Spearman rank agreement, reference top-10% recall, and selection regret. Under the shaped reference, this is

$$R_{\text{sel}} = c^* \left(\arg \min_{a \in \mathcal{A}} \hat{c}(a) \right) - \min_{a \in \mathcal{A}} c^*(a). \quad (3)$$

For the two contact tasks, we additionally report direct physical task-distance regret,

$$R_{\text{phys}} = d_{\text{task}} \left(\arg \min_{a \in \mathcal{A}} \hat{c}(a) \right) - \min_{a \in \mathcal{A}} d_{\text{task}}(a), \quad d_{\text{task}}(a) = \|o_T(a) - o_g\|_2. \quad (4)$$

Supplementary stateprobe diagnostics measure elite decode error, optimism, and a shuffled-residual baseline. Their definitions and results are in Appendix B.

To test adaptive refitting, we fork CEM from candidate-for-candidate identical first populations. One branch refits on $\hat{c}$ and the other on c^* . Subsequent samples use the same base Gaussian noise transformed by each branch’s fitted mean and variance. Both branches are evaluated at the same sequence of simulator snapshots, making the refitting cost the only intervention.

We also audit whether final CEM populations contain an exactly successful candidate. This supporting availability analysis is reported in Appendix B.

3.4 Evaluation splits and uncertainty

All reported MetaWorld evaluation splits are disjoint from the probe-training seeds. Population-level intervals are bootstrap confidence intervals clustered by environment seed; episode rates use Wilson intervals. Uncertainty is over environment seeds, not checkpoint-training seeds. Exact seed ranges, resampling counts, and per-cell testing details are in Appendix A.

3.5 OGBench-Cube audit

The OGBench extension uses a fixed manifest of 32 snapshots and the 300 final CEM candidates persisted for each snapshot. We compare four selectors on each population:

1. the released predicted-endpoint squared latent- L_2 cost;
2. squared latent L_2 after executing each candidate and encoding its true rendered endpoint, using the dataset goal image;
3. the same true-endpoint cost using a goal rendered from the exact future state in the same renderer; and
4. the physically best candidate by cube-to-goal distance.

The same-renderer true-endpoint arm is primary: it removes learned endpoint prediction and renderer mismatch while retaining the frozen encoder and terminal squared- L_2 rule. Physical regret is selected cube distance minus the best distance among the same 300 candidates. This isolates endpoint scoring on fixed populations; it does not rerun search. Full integrity checks and the evaluation protocol are in Appendix C.

MetaWorld implementation integrity. After matching renderer construction, image orientation, and goal-frame selection to the released stack, our Reach control is consistent with its reported performance (37.5% versus $44.8 \pm 8.9\%$) (Terver et al., 2026). Appendix A gives the complete protocol.

4 Results

4.1 Exact outcomes expose terminal-cost failures

Table 1 varies only the terminal cost after replacing the learned predictor with simulator rollout. The same finite search solves nearly all push episodes and most pick-place episodes with the reference cost. Both learned costs remain near failure.

Table 1: Held-out terminal-cost comparison on 64 fresh paired episodes per cell. Each task reports episode-end success and mean final object-to-goal distance in cm. Every row uses the same 100×6 CEM budget and exact simulator candidate rollouts.

	dynamics	repr.	cost	push		pick-place	
				success	dist.	success	dist.
simulator	—		state reference	64/64	0.56	49/64	2.20
simulator	DINO-WM		latent L_2	0/64	22.87	0/64	26.60
simulator	DINO-WM		stateprobe	5/64	19.70	0/64	23.67
simulator	JEPA-WM		latent L_2	0/64	22.87	0/64	26.68
simulator	JEPA-WM		stateprobe	1/64	20.99	1/64	24.19

The zero-success cells have a Wilson 95% upper bound of 5.7%; the reference rates have intervals [94.3, 100]% (push) and [64.9, 85.3]% (pick-place). All 64 push episodes and 49 pick-place episodes solved by the reference cost fail under latent L_2 . These are not near misses: latent L_2 ends 23–27 cm from the goal, versus task radii of 5 and 7 cm.

Learned-dynamics error is therefore not required for failure in this harness, and the fixed search budget is capable of solving the tasks when supplied an informative cost. Separate sensitivity axes extending to 24 refits and to 300 candidates per iteration do not rescue the contact costs (Appendix E). We next examine how the tested costs rank the candidates that CEM actually produces.

The failure is task-dependent. The physical cost solves all 16 episodes of button-press, drawer-close, and window-close. With the same exact outcomes and search budget, latent L_2 solves none of button-press or window-close but all of drawer-close (Table 2). Door-open and assembly are inconclusive because the physical control itself solves only 2/16 and 0/16; peg-insert-side failed the pre-specified expert-goal gate. The drawer result shows that latent distance can work when its visual ordering aligns with the task. For all breadth tasks, the state reference uses the generic goal-observable state slots specified in Appendix A.

Table 2: Pre-specified MetaWorld breadth test with exact candidate dynamics (16 paired seeds per task). Rows with a weak physical control are reported but not interpreted. For 16/16 and 0/16, Wilson 95% intervals are [80.6, 100]% and [0, 19.4]%, respectively. On each failure row passing the control, all 16 pairs are discordant (exact McNemar $p = 3.05 \times 10^{-5}$).

task	state reference	latent L_2	interpretation
button-press	16/16	0/16	additional failure
drawer-close	16/16	16/16	boundary: plannable
window-close	16/16	0/16	additional failure
door-open	2/16	0/16	excluded: weak control
assembly	0/16	0/16	excluded: failed control

4.2 Cost ordering deteriorates on search-induced candidates

We first evaluate latent L_2 on the populations generated by its own search. On fresh populations, it retains weak ordering on push ($\rho = 0.25, 0.27$) but essentially none on pick-place (0.02, −0.02). Its top-10% recall follows the same split, and its initial shaped-reference regret is 2.98–3.96 cm. After five refits, rank correlation is negative in all four cells and recall falls to 0.01–0.02. Thus pick-place fails before refitting; on push, weak initial ordering becomes anti-aligned as CEM updates its population.

Table 3: Latent L_2 on its own CEM populations: 16 episode seeds, seven replans per episode, 112 populations and 11,200 candidates per cell and stage. Initial and final denote CEM iterations 0 and 5. Recall is reference top-10% intersection-over-10, whose chance level is 0.10. R_{sel} is iteration-0 within-population shaped-reference regret in cm. Brackets are seed-clustered 95% intervals. Stateprobe values are included only as a compact secondary comparison; its full diagnostics are in Appendix D.

cell	ρ initial	ρ final	recall initial	recall final	initial R_{sel}
DINO push	0.25 [0.16, 0.32]	-0.08 [-0.14, -0.03]	0.16 [0.12, 0.19]	0.02 [0.01, 0.04]	2.98 [2.78, 3.21]
DINO pick	0.02 [-0.05, 0.10]	-0.09 [-0.14, -0.05]	0.07 [0.04, 0.11]	0.01 [0.01, 0.02]	3.74 [3.41, 4.09]
JEPA push	0.27 [0.19, 0.34]	-0.08 [-0.14, -0.02]	0.16 [0.12, 0.19]	0.02 [0.01, 0.04]	3.01 [2.71, 3.32]
JEPA pick	-0.02 [-0.09, 0.06]	-0.12 [-0.16, -0.08]	0.06 [0.03, 0.09]	0.01 [0.00, 0.02]	3.96 [3.60, 4.34]
<i>stateprobe, same estimands</i>					
range over 4 cells	0.43 to 0.55	0.11 to 0.16	0.19 to 0.26	0.07 to 0.10	2.05 to 2.48

Regret magnitudes should not be compared across iterations because refitting compresses the candidate set. The matched-refit experiment below tests whether these ordering failures change what search proposes next.

4.3 Misranking changes later candidates

We next fork CEM on a common matched-snapshot sequence from identical initial candidates. One branch updates on the learned cost and the other on the physical cost; sampling noise is shared. Table 4 reports the difference after five updates.

Table 4: Cost-only CEM intervention. Values are learned-cost minus physical-cost refitting for each branch’s best candidate after five updates; positive values favor physical-cost refitting. Brackets are seed-bootstrap 95% intervals. Stateprobe cells use $n = 8$ seeds; latent L_2 cells use $n = 16$ seeds.

cell	final Δ best task distance	final Δ best shaped cost
latent L_2 , DINO push	2.18 [1.86, 2.50] cm	3.87 [3.38, 4.32] cm
latent L_2 , DINO pick	1.13 [0.72, 1.55] cm	3.23 [2.86, 3.62] cm
stateprobe, DINO push	1.09 [0.62, 1.61] cm	1.64 [1.14, 2.20] cm
stateprobe, DINO pick	0.84 [0.32, 1.35] cm	1.51 [0.98, 2.06] cm
stateprobe, JEPA push	1.55 [1.22, 1.88] cm	2.01 [1.65, 2.35] cm
stateprobe, JEPA pick	1.14 [0.39, 1.98] cm	1.77 [0.96, 2.64] cm

Latent- L_2 refitting produces candidates 2.18cm worse on push and 1.13cm worse on pick-place in direct task distance; both confidence intervals exclude zero. The corresponding directions hold in 16/16 and 14/16 seed means, respectively. Thus the ranking failure is not merely descriptive: using latent L_2 to update CEM changes later proposals for the worse on both tested contact tasks.

Stateprobe as a secondary diagnostic. The stateprobe illustrates a distinct warning about average validation. On held-out expert trajectories, its scalar cost has broad rank agreement ($\rho = 0.80$ – 0.83), but reference top-10% recall is only 0.18–0.27. On CEM-generated populations, rank agreement falls from 0.43–0.55 to 0.11–0.16 and elite recall from 0.19–0.26 to 0.07–0.10. Stateprobe refitting also worsens later candidates by 0.84–1.55 cm (Table 4). Thus global held-out rank agreement does not certify the decision-relevant tail, and search can further degrade it. Full readout, selection, and shuffled-residual diagnostics are in Appendix D and B.

Final proxy-guided populations also contain exactly successful candidates only rarely; this supporting availability audit is reported in Appendix B. It is consistent with earlier refits degrading proposal quality, but does not separately identify all sources of scarcity.

4.4 A second encoder shows residual misranking

On the fixed OGBench populations, replacing the predicted endpoint with its rendered outcome reduces mean physical regret by 4.51 cm [1.84, 7.52] and raises success from 16/32 to 21/32 (Table 5). Dynamics error therefore matters. It is not the whole explanation: true-endpoint latent L_2 still leaves 3.29 cm [1.49, 5.67] regret and misses four successes found by the physical selector. Rank agreement improves from $\rho = 0.006$ to 0.395, but remains imperfect. Using the dataset goal or a goal rendered in the same world changes mean regret by only 0.03 cm.

Table 5: OGBench-Cube outcome audit on 32 independent snapshots with 300 fixed final CEM candidates each. Distance and regret are in cm; brackets are 95% snapshot-bootstrap intervals. ρ is the mean within-snapshot Spearman agreement between selector cost and physical cube distance over 28 non-degenerate snapshots. The physical-best row reports the best candidate available in each population.

selector	success	selected dist.	physical regret		ρ
predicted endpoint L_2	16/32	12.11	7.80 [4.86, 10.96]	0.006 [-0.025, 0.037]	
true endpoint, dataset goal	21/32	7.63	3.32 [1.54, 5.70]	0.396 [0.234, 0.536]	
true endpoint, rendered goal	21/32	7.60	3.29 [1.49, 5.67]	0.395 [0.236, 0.541]	
physical best in population	25/32	4.31	0		—

This result concerns one LeWM encoder, one Cube task, and fixed final populations; the MetaWorld experiment provides the direct refitting test. Replay and renderer integrity checks are reported in Appendix C.

5 Discussion and Limitations

The experiments reveal two ways a cost can fail. Latent L_2 carries little task-relevant ordering on fresh pick-place candidates and only weak ordering on push. Stateprobe begins with better ordering, despite being imperfect, but CEM moves into regions where that ordering degrades. In both cases, updating CEM with the learned cost produces worse later candidates than updating it with the physical cost. The OGBench result adds an important qualification: dynamics error can be a large part of total regret even when cost misranking remains after that error is removed.

The practical implication is simple. A world model should not be evaluated only by average prediction error, and a planning representation should not be evaluated only on held-out trajectories. When simulator access permits, execute the planner’s candidates, score the same outcomes with both learned and physical costs, and change only the cost used to update search. These tests indicate whether effort should go toward the predictor, the terminal objective, or candidate generation.

Our experiments cover two MetaWorld contact tasks with one DINOv2 representation, a cost-only refitting intervention run on eight seeds per stateprobe cell and sixteen seeds per latent- L_2 cell, and one OGBench-Cube task with an independently trained encoder. Simulator-state costs and probe labels are evaluation tools, not deployable inputs. Representation training seeds, alternative probe architectures, and physical robots remain untested. Non-CEM optimizers are tested only as planner-level robustness controls; candidate-level ranking and matched-refit analyses remain CEM-specific. The shuffled-score result separates generic error magnitude from error structure for the shaped planning cost but not for direct task distance; that separation for task distance is left open.

6 Conclusion

A controlled audit that holds the encoder, terminal cost, and search budget fixed while replacing learned prediction with exact simulator outcomes reveals a failure that survives perfect dynamics: in MetaWorld, the same finite search succeeds with a physical cost while latent L_2 and a supervised state-space cost remain near failure. The reason is not just inaccurate outcome prediction – the costs misrank exact outcomes, and CEM acts on that ranking when it generates later candidates. OGBench-Cube shows the same residual scoring problem with a distinct, independently trained encoder, while also confirming that dynamics error

can explain a substantial share of total regret. The drawer-close success shows that the failure is task-dependent rather than universal. The broader lesson is to test the ordering a cost induces on the candidates created by search, rather than infer planning quality from average held-out accuracy.

Broader Impact Statement

This work studies failure modes of simulation-based robot world-model planning and may support more reliable evaluation of learned planning systems. The experiments do not establish real-world robot safety or generality beyond the evaluated tasks, checkpoints, costs, and planners. Overgeneralizing the findings could create misplaced confidence in an audit or unwarranted conclusions about representation learning more broadly. Independent simulator and physical-system validation, together with deployment-specific safety controls, remains necessary before applying these conclusions to robots.

References

- Benjamin Eysenbach, Tianjun Zhang, Sergey Levine, and Ruslan Salakhutdinov. Contrastive learning as goal-conditioned reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/e7663e974c4ee7a2b475a4775201ce1f-Abstract-Conference.html.
- Amir-massoud Farahmand. Iterative value-aware model learning. In *Advances in Neural Information Processing Systems*, 2018.
- Amir-massoud Farahmand, André Barreto, and Daniel Nikovski. Value-aware loss function for model-based reinforcement learning. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pp. 1486–1494, 2017.
- Baoqi Gao, Ruize Han, Miao Wang, and Song Wang. IMWM: Intuition models complement world models for latent planning. *arXiv preprint arXiv:2606.01626*, 2026.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, 2023.
- Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, 2019.
- Jacek Karwowski, Oliver Hayman, Xingjian Bai, Klaus Kiendlhofer, Charlie Griffin, and Joar Skalse. Goodhart’s law in reinforcement learning. In *International Conference on Learning Representations*, 2024.
- Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Learning for Dynamics and Control*, 2020.
- Liangyu Li, Shengzhi Wang, and Qingwen Liu. Beyond Euclidean proximity: Repairing latent world models with horizon-matched trajectory reachability metrics. *arXiv preprint arXiv:2605.22164*, 2026a.
- Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models. *arXiv preprint arXiv:2605.07278*, 2026b.
- Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation. *arXiv preprint arXiv:2605.21800*, 2026a. URL <https://arxiv.org/abs/2605.21800>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026b. URL <https://arxiv.org/abs/2603.19312>.

- Vivek Myers, Chongyi Zheng, Anca Dragan, Sergey Levine, and Benjamin Eysenbach. Learning temporal distances: Contrastive successor features can provide a metric structure for decision-making. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 37076–37096, 2024. URL <https://proceedings.mlr.press/v235/myers24a.html>.
- Hoang Nguyen, Xiaohao Xu, and Xiaonan Huang. Latent geometry beyond search: Amortizing planning in world models. *arXiv preprint arXiv:2605.08732*, 2026. URL <https://arxiv.org/abs/2605.08732>.
- Evgenii Nikishin, Romina Abachi, Rishabh Agarwal, and Pierre-Luc Bacon. Control-oriented model-based reinforcement learning with implicit differentiation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=M992mjgKzI>.
- Arjun Parthasarathy, Nimit Kalra, Rohun Agrawal, Yann LeCun, Oumayma Bounou, Pavel Izmailov, and Micah Goldblum. Closing the train-test gap in world models for gradient-based planning. *arXiv preprint arXiv:2512.09929*, 2025.
- Reuven Y. Rubinstein. Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1):89–112, 1997.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models. *arXiv preprint arXiv:2607.02403*, 2026.
- Joar Skalse, Nikolaus Howe, Dmitrii Krashenninikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.
- James E. Smith and Robert L. Winkler. The optimizer’s curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006. doi: 10.1287/mnsc.1050.0451.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*, 2026. URL <https://openreview.net/forum?id=cHZn5Gdh8e>.
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning*, 2020.
- Yang Yu, Shiyuan Zhang, Yifei Sheng, Haoxiang Ren, and Haoxin Lin. How should world models be evaluated for embodied decision-making? a decision-making-centric position. *arXiv preprint arXiv:2606.15032*, 2026.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 79115–79135, 2025. URL <https://proceedings.mlr.press/v267/zhou25t.html>.

A Audit Protocol and Reproducibility

Candidate execution. For every candidate action sequence, the harness restores an identical MuJoCo snapshot, executes the sequence, renders the endpoint, and runs the released encoder on that rendered observation. Simulator state is used only for evaluation, stateprobe supervision, and the physical reference branch; it is never supplied to the learned predictor.

Table 6: Fixed MetaWorld protocol used by the exact-dynamics and candidate- level experiments.

item	implementation
CEM distribution	4-D raw actions; initial mean 0 and variance 1; actions clipped to $[-1, 1]$; no warm start between replans
Refitting	100 candidates, six iterations, 10 elites; full elite mean/variance update, no exponential smoothing; variance floor 10^{-4}
Horizon and MPC	$H = 6$ model steps; frame skip 5 gives 30 raw simulator actions per candidate; execute 3 model steps (15 raw actions) before replanning; horizon shrinks near episode end
Episode protocol	one zero-action environment warmup; at most 100 subsequent controlled raw actions; strict final-step simulator success flag
Environment and view	MetaWorld V3 goal-observable tasks; <code>corner2</code> camera at (0.75, 0.075, 0.7); 224×224 render; renderer reconstructed after camera assignment; vertical image flip
Model inputs	scripted expert’s final frame as goal; released visual preprocessing and released action normalization
Breadth reference	All five breadth tasks use $h = \text{state}[0:3]$ and $o = \text{state}[4:7]$; $o_g = \text{goal_state}[4:7]$ comes from the scripted expert’s final observation. The reference applies Equation (2) with these generic goal-observable slots
Object probe	two-layer, six-head transformer with learned query, hidden size 512, 2.57M parameters; Adam, learning rate 5×10^{-4} , batch 64, 8 epochs; final epoch used
End-effector probe	mean/max-pooled MLP, hidden size 512, 0.53M parameters; Adam, learning rate 10^{-3} , batch 64, 3 epochs; final epoch used
Probe data	12,312 expert transitions for training and 1,368 for validation, split by trajectory 90/10 with seed 0; 240 random-action frames from both contact tasks (seeds 20000–20007, 60 steps, sampled every 4 steps); off-policy-to-expert batch ratio 0.5
Cost coefficient	hand-object weight fixed at 0.5 in both stateprobe and reference costs before the confirmatory and candidate-level evaluation; not tuned on the reported evaluation seeds
Uncertainty	5,000 seed-clustered bootstrap resamples by default; 10,000 for task-aligned and OG-Bench estimates; 1,000 within-population residual permutations for shaped regret and 5,000 for task-distance regret; Wilson intervals and exact paired McNemar tests for episode rates

Immediate selection. The first-population audit evaluates 100 sequences per seed before any CEM refit. The top-10% elite sets are independently formed under stateprobe and reference cost. The shuffled-residual baseline performs 1,000 within-population permutations. Seed-clustered percentile intervals use 5,000 bootstrap resamples.

Held-out scalar cost. We reconstruct the probes’ immutable trajectory split (90/10, seed 0) without loading or changing the world model. For each non-Reach held-out expert trajectory, every frame-skip-5 endpoint is scored against that trajectory’s final latent goal using the deployed stateprobe composition and against the final simulator-state goal using c^* . Metrics first reduce within trajectory; the primary 95% intervals resample the 11 push/pick-place trajectories; the all-manipulation aggregate uses all 67 non-Reach trajectories. This test uses one probe-training seed and expert trajectories; the CEM population test measures the distribution encountered during planning.

Adaptive branch. Both CEM branches share the simulator snapshot, first candidate population, and base Gaussian noise. Each branch transforms later common noise by its own mean and variance. The physical-cost branch determines the common sequence of MPC snapshots, so the comparison isolates search updates at those snapshots.

Candidate availability. Candidate-horizon success is evaluated with the simulator’s exact task predicate. It need not imply episode-end success after subsequent MPC replans. Intervals cluster replans by environment seed. The final populations reflect all earlier proxy-guided updates.

Table 7: Held-out expert-trajectory validation for the fixed stateprobe. Channel metrics use all 1,368 transitions; scalar metrics use 209 transitions from the 11 held-out push and pick-place trajectories. Channel RMSE is per coordinate; channel medians and scalar MAE are cm. Scalar ρ is the mean within-trajectory Spearman coefficient, and recall is reference top-10% recall; brackets are trajectory-bootstrap 95% intervals.

repr.	object RMSE	object med.	hand RMSE	hand med.	scalar MAE		scalar ρ		recall
DINO-WM	1.71	2.09	4.31	6.65	2.43	[1.97, 2.98]	0.800	[0.700, 0.890]	0.18 [0.00, 0.41]
JEPA-WM	1.64	2.10	3.75	5.62	2.84	[2.36, 3.41]	0.826	[0.756, 0.890]	0.27 [0.09, 0.50]

B Supplementary Stateprobe and Availability Diagnostics

For a population $\mathcal{A}$ scored by stateprobe and simulator reference, the shaped-reference selection regret is

$$R_{\text{sel}} = c^* \left(\arg \min_{a \in \mathcal{A}} \hat{c}(a) \right) - \min_{a \in \mathcal{A}} c^*(a). \quad (5)$$

Let $e_o(a) = \|\hat{o}_T(a) - o_T(a)\|_2$ and let $E_{\hat{c}}$ denote the ten smallest-stateprobe-cost candidates. We report object-only elite error excess $\Delta_{\text{err}} = \text{mean}_{a \in E_{\hat{c}}} e_o(a) - \text{mean}_{a \in \mathcal{A}} e_o(a)$ and optimism enrichment Δ_{opt} , defined analogously for $o(a) = c^*(a) - \hat{c}(a)$. The shuffled-residual baseline writes $\hat{c}(a) = c^*(a) + r(a)$ and permutes residuals within each population before selection; it preserves the residual magnitudes while breaking their association with candidates.

Table 8: Stateprobe immediate-selection diagnostics on identical iteration-0 populations: 16 independent episodes, seven replans per episode, 112 populations, and 11,200 candidates per cell. Values are cm; brackets are seed-clustered 95% confidence intervals. The null permutes stateprobe residuals within each population; actual-minus-null intervals quantify seed uncertainty.

cell	Δ_{err}		Δ_{opt}		actual R_{sel}	null R_{sel}	actual – null	
DINO push	1.27	[0.87, 1.64]	2.33	[2.08, 2.59]	2.48	[2.26, 2.69]	1.12	1.36 [1.16, 1.55]
DINO pick	1.25	[0.79, 1.76]	2.16	[1.81, 2.56]	2.25	[1.86, 2.64]	1.07	1.17 [0.83, 1.54]
JEPA push	1.24	[0.64, 1.83]	2.28	[1.88, 2.69]	2.05	[1.81, 2.32]	1.13	0.92 [0.72, 1.13]
JEPA pick	1.40	[0.88, 1.94]	2.21	[1.90, 2.59]	2.06	[1.76, 2.34]	1.15	0.91 [0.66, 1.15]

Selection over any noisy score creates shaped-cost regret: the shuffled baseline incurs 1.07–1.15 cm, while the actual score incurs roughly twice as much. The excess is positive in all four cells. Under direct object-to-goal distance, however, actual-minus-shuffled regret ranges only from -0.06 to 0.08 cm and excludes zero in one cell. The residual result therefore characterizes the operational shaped cost, not a consistent excess in immediate task-distance regret.

Candidate availability. The final CEM population contains at least one exactly successful sequence on 8.0% and 3.6% of DINO push and pick-place replans, respectively, and on none of the corresponding JEPA replans. These populations reflect earlier proxy-guided refits and therefore do not independently measure initial proposal coverage.

Table 9: Final-iteration exact-success availability and selection over 112 replans from 16 episodes per cell. Brackets are seed-clustered 95% intervals; physical regret is based on object-to-goal distance.

cell	exact-success availability	selected success	physical regret
DINO push	8.0% [0.0, 17.0]	6.3% [0.0, 14.3]	0.41 [0.25, 0.57] cm
DINO pick	3.6% [0.0, 8.9]	1.8% [0.0, 5.4]	0.44 [0.24, 0.67] cm
JEPA push	0.0%	0.0%	0.42 [0.29, 0.55] cm
JEPA pick	0.0%	0.0%	0.59 [0.42, 0.75] cm

The two zero cells have degenerate cluster bootstraps, so only their observed rates are shown. In the matched-snapshot stateprobe intervention, switching to physical refitting increases availability by 2.5–13.1 percentage points; only the JEPA-push interval excludes zero.

C OGBench-Cube Protocol and Integrity

The OGBench audit uses the released `quentin11/lewm-cube` checkpoint and the official `ogbench/cube_single_expert.h5` dataset, with goal offset 25, horizon 5, action block 5, and 300 persisted final candidates for each of 32 snapshots. The manifest fixes one snapshot per distinct episode. A candidate is successful when its final cube-to-goal distance is below 4cm. Snapshot is the unit of uncertainty, and all reported intervals use 10,000 paired bootstrap draws.

The evaluator creates two independent `World/MjModel` instances. Each is initialized once with visual variation disabled, after which camera, light, material, texture, and geometry signatures must agree. Before each state render or candidate rollout, it calls `mj_resetData`, restores dataset position and velocity plus the future cube target, calls `mj_forward`, aligns finite-difference previous-state fields, and disables solver warm-start. The two worlds must agree within 10^{-5} m on physical distance and exactly on success, executed steps, endpoint pixels, and selected index; encoder costs must agree within 10^{-5} . All 32 shards pass, with zero worst-case repeat error on the reported physical, pixel, latent-cost, and selection fields.

For an exact initial or goal state, the encoder-domain ratio is the larger same-state squared-latent discrepancy between evaluator and dataset render, divided by the smaller initial-to-goal separation in the two render domains. The precommitted maximum is 0.25; the worst observed ratio is 6.59×10^{-4} . The dataset-goal and same-renderer-goal controls select the same number of successes and differ by 0.0315 cm in mean regret. These checks establish reproducible, encoder-near-domain endpoint evaluation.

D Probe and Cost Details

The stateprobe decodes object and end-effector positions from frozen endpoint tokens. The object readout uses a two-layer transformer with a learned query token; the end-effector readout is a mean/max-pooled MLP. Both are trained with squared error on simulator-labelled off-policy frames, and no encoder parameters are updated. The stateprobe cost applies the same object-goal plus hand-object functional form as the simulator-state reference, comparing the decoded endpoint object with the decoded goal-frame object. Consequently, the audit diagnoses the full representation-readout-cost composition and cannot distinguish information absent from the encoder from information inaccessible to this readout. Held-out expert-trajectory errors for the exact probes are reported in Table 7.

Table 10: Stateprobe behavior on optimizer-induced candidates. Initial and final denote CEM iterations 0 and 5. Each row contains 112 populations and 11,200 candidates from 16 episode seeds; brackets are seed-clustered 95% intervals. Object and hand columns are mean Euclidean endpoint errors in cm. ρ compares stateprobe and reference shaped costs; reference top-10 recall is intersection-over-10 of their elite sets.

repr.	task	stage	object error		hand error		cost ρ		reference top-10 recall	
DINO	push	initial	5.18	[4.76, 5.61]	6.39	[5.76, 7.09]	0.43	[0.36, 0.49]	0.19	[0.16, 0.22]
DINO	push	final	6.72	[5.99, 7.43]	6.60	[5.76, 7.50]	0.11	[0.05, 0.16]	0.07	[0.04, 0.11]
DINO	pick	initial	4.98	[4.37, 5.57]	6.16	[5.41, 7.03]	0.45	[0.37, 0.52]	0.22	[0.19, 0.27]
DINO	pick	final	6.35	[5.32, 7.35]	6.27	[5.28, 7.43]	0.12	[0.07, 0.17]	0.08	[0.06, 0.11]
JEPA	push	initial	4.79	[4.08, 5.51]	8.52	[8.13, 9.03]	0.52	[0.47, 0.57]	0.26	[0.21, 0.30]
JEPA	push	final	6.13	[4.93, 7.34]	8.70	[8.20, 9.33]	0.15	[0.08, 0.22]	0.08	[0.05, 0.12]
JEPA	pick	initial	4.78	[4.15, 5.42]	8.26	[7.92, 8.60]	0.55	[0.52, 0.58]	0.26	[0.22, 0.30]
JEPA	pick	final	6.15	[5.11, 7.15]	8.38	[8.02, 8.74]	0.16	[0.12, 0.21]	0.10	[0.07, 0.12]

The expert split shows that the object readout is substantially more accurate than the hand readout. On the CEM populations actually used by the mechanism test, initial cost rank agreement is moderate ($\rho = 0.43$ – 0.55), but final-population agreement is only 0.11 – 0.16 and reference top-10 recall is 0.07 – 0.10 . These measurements characterize the fixed stateprobe used in planning.

The probe weights use random-rollout episodes from environment seeds 20000–20007 as training augmentation. Tables 7 and 10 use the held-out trajectory split and disjoint CEM evaluation seeds. The held-out terminal-cost comparison (30000–30063), selection audit (41000–41015), branch intervention (42000–42007) stateprobe, 42000–42015 latent- L_2 push, 44000–44015 latent- L_2 pick-place), and availability audit (40000–40015) use disjoint environment seeds.

E Search-Budget Sensitivity

A separate DINO-WM sensitivity run varies CEM refitting iterations on 16 fixed episodes (seeds 10000–10015) while keeping 100 candidates per iteration. Table 11 shows that increasing refits fourfold beyond the main setting does not rescue either contact cost, whereas the latent- L_2 Reach control saturates at 16/16 by six refits. At 12 refits, a separate population-size sweep over 50, 100, and 300 candidates gives push success 0/8, 0/8, and 0/8 for latent L_2 , and 0/8, 1/8, and 0/8 for stateprobe. Thus the failure persists across the tested budgets.

Table 11: Episode-end success as CEM refitting iterations increase; each iteration contains 100 candidates and each cell has 16 paired episodes.

task	cost	2 refits	6 refits	12 refits	24 refits
push	latent L_2	0/16	0/16	0/16	1/16
push	stateprobe	0/16	1/16	1/16	2/16
pick-place	latent L_2	0/16	0/16	0/16	0/16
pick-place	stateprobe	0/16	0/16	0/16	1/16
reach	latent L_2	5/16	16/16	16/16	16/16

F Planner-Level Robustness Control

On DINO-WM push under oracle dynamics, stateprobe remains near failure with CEM (1/16), MPPI (0/16), and random shooting (2/16); latent L_2 remains near failure with MPPI (0/16) and random shooting (0/16). Their free-space Reach controls are 16/16, 13/16, and 6/16, respectively. We additionally run a privileged physical-reference cost on push itself, at the same seeds and budget, under every tested optimizer: it solves 16/16 under CEM, 16/16 under MPPI, and 11/16 under random shooting. Because this physical reference converts the same finite search into success under all three optimizers – including a majority of shooting’s non-adaptive, fixed-proposal search – the near-failure of stateprobe and latent L_2 on push is not explained by planner capacity alone. The candidate-level and refitting analyses in the main paper use CEM.